

Name : Pushpak Shende
DS-C-15 CRT Batch B1

POST-TEST

1. Output :-

Constructor called
Init called

Constructor called – Spring creates bean object first.

Init called – `@PostConstruct` runs immediately after dependency injection and bean creation

2. It returns only status code no body no message
return new ResponseEntity<>(u , [HttpStatus.OK](#));

3.

StudentDTO

```
package com.crt2.posttest.dto;
```

```
import jakarta.validation.constraints.Min;  
import jakarta.validation.constraints.NotBlank;  
import lombok.*;
```

```
@Getter
```

```
@Setter
```

```
@AllArgsConstructor
```

```
@NoArgsConstructor
```

```
public class StudentDTO {
```

```
    @NotBlank(message = "name should be there")  
    private String name;
```

```
    @Min(18)  
    private int age;
```

```
}
```

```

StudentController
package com.crt2.posttest.controller;

import com.crt2.posttest.dto.StudentDTO;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    @PostMapping
    public ResponseEntity<String> createStudent(
        @Valid @RequestBody StudentDTO student , BindingResult result
    ){
        if (result.hasErrors()) {
            return ResponseEntity.badRequest()
                .body(result.getFieldError().getDefaultMessage());
        }
        return ResponseEntity.status(HttpStatus.CREATED)
            .body("Student created Successfully");
    }
}

```

4.

@Transactional

– This method updates two accounts
 If one save succeeds and second fails

- Money deducted to sender
- Money not added to receiver

Without transaction – inconsistent data

With transaction – all operations succeed OR rollback completely

5.

Employee entity

```
package com.crt2.posttest.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@Entity
@Getter
@Setter
@NoArgsConstructor
@ToString
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double salary;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

}
```

Department Entity

```
import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

import java.util.List;

@Getter
@Setter
@NoArgsConstructor
@ToString
@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int dId;

    private String departmentName;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}
```

6.

a)

Filter reads JWT token from Authorization header

Validates token , extracts username , and sets authentication in Spring Security context.

b)

String token = header.substring(6); – issue is here

“Bearer “ – 7 characters there a space after Bearer

String token = header.substring(7); // correct code

7. Write a Spring Data JPA repository method using `@Query` to find all Product entities where price is less than a given value and category matches a given string. Use named parameters .

```
package com.crt2.posttest.que7;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;

public interface ProductRepository extends JpaRepository<Product, Long> {

    @Query("SELECT p FROM Product p WHERE p.price < :price AND p.category = :category")
    List<Product> findProducts(@Param("price") double price,
                              @Param("category") String category);
}
```

8. Explain the N+1 query problem in JPA in your own words with a simple example. What is the most common cause? Write one line of code that fixes it in a `@Query` method using JOIN FETCH.

N + 1 query problem

When one query fetches parent entities and additional queries fetch child entities individually

– causes Lazy loading

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("SELECT o FROM Order o JOIN FETCH o.items")
    List<Order> findAllWithItems();
}
```

9. Write a SQL query to find the second highest salary from a table employees(id, name, salary). Use a subquery approach. Then write the same logic as a JPQL @Query method in a JPA repository.

```
package com.crt2.posttest.que9;

import com.crt2.posttest.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT MAX(e2.salary) FROM Employee e2)")
    Double getSecondHighestSalary();
}
```

10. Given a Student entity with fields name, email, and city, write FOUR Spring Data JPA derived query method names (no @Query) for the following: (a) find by city, (b) find by name and city, (c) find by email containing a keyword, (d) count students in a city. Write the method signatures only.

```
package com.crt2.posttest.que10;

import com.crt2.posttest.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface StudentRepository extends JpaRepository<Student, Long> {

    List<Student> findByCity(String city);

    List<Student> findByNameAndCity(String name, String city);

    List<Student> findByEmailContaining(String keyword);

    long countByCity(String city);
}
```

11. A developer runs the following code to load 10 orders and print their items. console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List orders = orderRepo.findAll(); for (Order o : orders) {  
    System.out.println(o.getItems().size()); }
```

`o.getItems().size();`

Because of this there's lazy loading

Change in repository and then update service

// repository

```
public interface OrderRepository extends JpaRepository<Order, Long> {
```

```
    @Query("SELECT o FROM Order o JOIN FETCH o.items")
```

```
    List<Order> findAllWithItems();
```

```
}
```

//Service

```
List<Order> orders = orderRepo.findAllWithItems();
```

```
for (Order o : orders) {
```

```
    System.out.println(o.getItems().size());
```

```
}
```

12. Write the application.yml configuration and the main class annotation needed to register a Spring Boot service named product-service with a Eureka server at <http://localhost:8761/eureka>.

application.yml

server:

port: 8081

spring:

application:

name: product-service

eureka:

```
client:
  service-url:
    defaultZone: http://localhost:8761/eureka
```

Main class

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
```

```
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

13. What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using RestTemplate and applies a Resilience4j @CircuitBreaker with a fallback method that returns a default PaymentResponse object.

Circuit Breaker prevents repeated calls to a failing service.

Why needed:

- avoids cascading failures
- improves system stability
- returns fallback response when service is down

```
// PaymentResponse
```

```
@Getter
@Setter
@NoArgsConstructor
@ToString
public class PaymentResponse {

    private String status;
```



```

    private String message;

}

// Payment Service
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class PaymentService {

    @Autowired
    private RestTemplate restTemplate;

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResponse callPaymentService() {
        return restTemplate.getForObject(
            "http://payment-service/pay",
            PaymentResponse.class
        );
    }

    public PaymentResponse paymentFallback(Exception ex) {
        return new PaymentResponse("FAILED", "Payment service unavailable");
    }
}

```

14. What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs

API Gateway is a single entry point for all client requests.

Benefits:

- routing requests
- centralized security
- load balancing
- hides internal services

application.yml

server:

port: 8080

spring:

application:

name: api-gateway

cloud:

gateway:

routes:

- id: order-service

uri: lb://order-service

predicates:

- Path=/api/orders/**

- id: user-service

uri: lb://user-service

predicates:

- Path=/api/users/**

15. What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically

Service Discovery allows services to find each other using service names instead of hardcoded IP addresses.

It is used because IPs can change

- Services scale dynamically
- Easier communication

Tools used :

Netflix Eureka

Config Class

```
@Configuration
public class AppConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

16. You have a monolithic Spring Boot app with modules: UserModule, OrderModule, ProductModule. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

a) Benefits

1. Independent deployment
2. Better scalability
3. Easier maintenance

(b) Challenges

1. Complex communication between services
2. Distributed debugging / monitoring

c) Eureka + API Gateway

Flow:

React frontend calls:

`http://localhost:8080/api/users`

API Gateway receives request → asks Eureka where user-service forwards request to correct instance.

Same for: order-service and product-service

So frontend uses one URL only, not multiple service URLs

17. Write a JUnit 5 + Mockito unit test for the getByld() method below. Your test should: (a) use @Mock and @InjectMocks, (b) stub the repository to return a sample Product, and (c) assert that the returned product name equals "Laptop"

```
import org.junit.jupiter.api.Test;

import org.junit.jupiter.api.extension.ExtendWith;

import org.mockito.InjectMocks;
import org.mockito.Mock;

import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)

public class ProductServiceTest {

    @Mock
    private ProductRepository repo;

    @InjectMocks
    private ProductService service;

    @Test
    void testGetByld() {

        Product product = new Product(1L, "Laptop");

        when(repo.findByld(1L)).thenReturn(Optional.of(product));

        Product result = service.getByld(1L);

        assertEquals("Laptop", result.getName());
    }
}
```

```
}  
}
```

18. What is the difference between `@Mock` (Mockito) and `@MockBean` (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses `@MockBean` correctly with `@SpringBootTest`.

`@Mock`

- Mockito annotation
- used in unit tests
- does not load Spring context

`@MockBean`

- Spring Boot test annotation
- replaces bean inside Spring context

Use:

- `@Mock` – unit testing
- `@MockBean` – Integration Testing

```
import org.springframework.boot.test.context.SpringBootTest;  
import org.springframework.boot.test.mock.mockito.MockBean;
```

```
@SpringBootTest  
public class ProductControllerTest {  
  
    @MockBean  
    private ProductService productService;  
}
```

19.

FROM eclipse-temurin:21-jre

COPY target/app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

Build command

```
docker build -t myapp:latest .
```

Run command

```
docker run -p 8080:8080 myapp:latest
```

20. Write a docker-compose.yml with two services: (a) db — using the postgres:15 image with environment variables for DB name, user, and password, and (b) app — using the myapp:latest image, mapping port 8080, connecting to the db service, and starting only after db starts using depends_on.

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  db:
```

```
    image: postgres:15
```

```
    environment:
```

```
      POSTGRES_DB: mydb
```

```
      POSTGRES_USER: postgres
```

```
      POSTGRES_PASSWORD: password
```

```
  app:
```

```
    image: myapp:latest
```

```
    ports:
```

```
      - "8080:8080"
```

```
    depends_on:
```

```
      - db
```

21. Write a JavaScript async function loadUsers() that fetches data from GET /api/users using the fetch API, checks if the response is successful (response.ok), and either renders a list of user names into a element or shows an error message in a element.

script .js

```
async function loadUsers() {
```

```
  const userList = document.getElementById("user-list");
```

```
  const error = document.getElementById("error");
```

```
  try {
```

```
    const response = await fetch("/api/users");
```

```

    if (!response.ok) {
      throw new Error("Failed to fetch users");
    }

    const users = await response.json();

    userList.innerHTML = "";

    users.forEach(user => {
      const li = document.createElement("li");
      li.textContent = user.name;
      userList.appendChild(li);
    });

  } catch (err) {
    error.textContent = err.message;
  }
}

```

22. The React component below has a bug that causes it to fetch data in an infinite loop. (a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

`useEffect()` has no dependency array.

Every render:

- fetch runs
- state updates
- rerender happens
- fetch runs again

Infinite loop.

```
import React, { useEffect, useState } from "react";
```

```

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/users")

```

```
        .then(res => res.json())
        .then(data => {
            setUsers(data);
            setLoading(false);
        });
    }, []);

    if (loading) {
        return <p>Loading...</p>;
    }

    return (
        <ul>
            {users.map(user => (
                <li key={user.id}>{user.name}</li>
            ))}
        </ul>
    );
}

export default UserList;
```